

ASCEND Autonomous Feature Recognition Pipeline

Compact System Design Reference

1. Goal

Design a lightweight onboard ML pipeline for the ISRO ASCEND mission.

The drone must:

- autonomously survey the arena
- detect terrain features
- compare them against organizer-provided seed images
- identify matching features
- record GPS coordinates
- complete the mission fully autonomously

System is optimized for:

- very small reference dataset (5–7 images per class)
 - embedded GPU compute
 - real-time inference onboard UAV
 - high robustness under terrain variation
-

2. Core Design Philosophy

The system separates the problem into two independent stages:

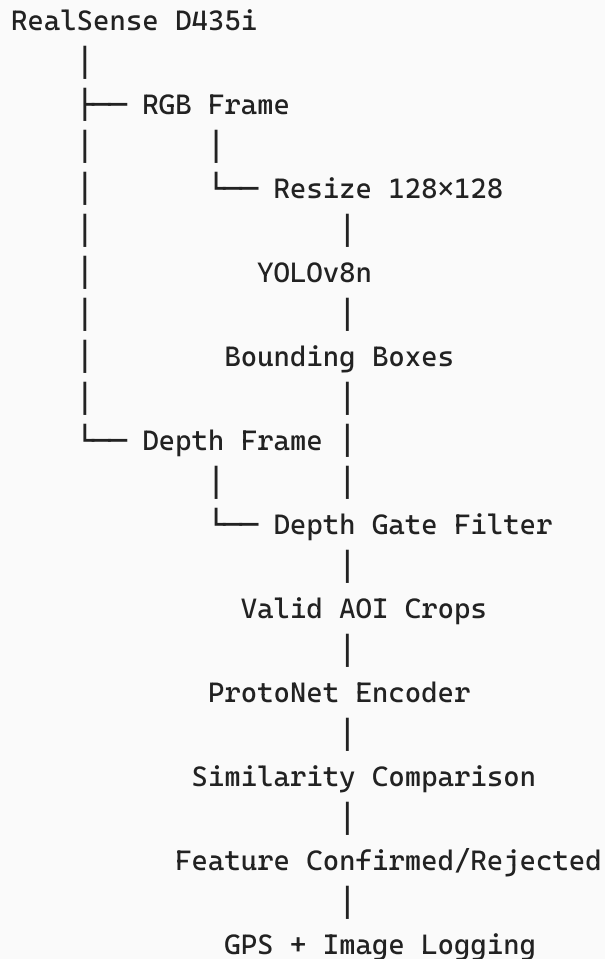
Stage	Responsibility
Stage 1	Find possible feature locations
Stage 2	Verify which feature it actually is

This is intentionally not an end-to-end detector.

Reason:

- Few-shot classification and object localization are different problems
 - Detection requires spatial search
 - Few-shot learning requires similarity comparison
 - Separating them improves stability, modularity, and runtime efficiency
-

3. High-Level Pipeline



4. Hardware Platform

Compute

- Jetson Orin Nano 8GB
- CUDA + TensorRT FP16 inference
- Shared RAM architecture
- No DLA available

Camera

- Intel RealSense D435i
- RGB: 1280×720 @ 30 FPS
- Depth: 848×480 @ 30 FPS

Runtime Assumptions

- Real-time onboard inference
 - No cloud compute
 - All models fully resident onboard
 - Fixed batch size = 1
-

5. Stage 1 — Detection

Model

YOLOv8n FP16 TensorRT engine.

Purpose

Used as:

- region proposal system
- saliency detector
- candidate feature extractor

Not used as final classifier.

Input

- RGB frame resized to 128×128

Output

- bounding boxes
- objectness scores

Why YOLOv8n

- very small model

- fast TensorRT inference
 - reliable spatial localization
 - fits embedded latency budget
-

6. Depth Gate Filter

Runs after YOLO.

Purpose:

remove false detections caused by:

- walls
- obstacles
- clutter
- multi-depth regions

Method:

- evaluate depth consistency inside bounding box
- compute:

$$\text{depth_variation} = \sigma_z / \text{mean}_z$$

Decision:

- low variation $\rightarrow$ flat terrain $\rightarrow$ accept
- high variation $\rightarrow$ reject

Runs on CPU concurrently with GPU inference.

7. Stage 2 — Few-Shot Classification

Model

ProtoNet with MobileNetV3-Small encoder.

Purpose

Determine which organizer feature the crop matches.

Why Few-Shot Learning

Only 5–7 seed images are available per feature.

Traditional classifiers overfit heavily in this regime.

ProtoNet instead:

- learns embedding similarity
 - compares image geometry in feature space
 - works well with extremely low data
-

8. Prototype Construction

Before flight:

1. Load organizer seed images
2. Apply augmentation
3. Generate embeddings
4. Average embeddings per class
5. Store prototype vectors

Result:

- one prototype vector per feature class

These remain resident in GPU memory during mission runtime.

9. Runtime Classification

For every accepted crop:

```
crop → encoder → embedding  
embedding → cosine distance to prototypes  
nearest prototype = predicted class
```

Decision:

- distance below threshold → confirmed feature
- otherwise rejected

10. Runtime Architecture

Thread Layout

Thread	Purpose
T1	Camera capture
T2	YOLO + depth filter
T3	ProtoNet inference
T4	Logging + storage

GPU Strategy

- TensorRT FP16
- fixed input shapes
- asynchronous memory transfer
- persistent prototype tensors
- stream overlap where possible

11. Latency Goals

Operation	Target
YOLO	~18 ms
Depth Gate	~2 ms
ProtoNet per crop	~8 ms
Total	~20–35 FPS effective

ProtoNet only runs when YOLO produces valid regions.

Most frames skip Stage 2 entirely.

12. Dataset Structure

```
datasets/  
├─ yolo_detection/  
├─ protonet_episodes/  
├─ seeds/  
└─ mission_log/
```

YOLO Dataset

Contains:

- bounding boxes
- terrain images
- synthetic augmentations

ProtoNet Dataset

Contains:

- texture classes
- no bounding boxes required
- episodic training structure

13. Key Design Decisions

Decision	Choice
Detector	YOLOv8n
Classifier	ProtoNet
Encoder	MobileNetV3-Small
Precision	FP16
Runtime	TensorRT
Input Strategy	Low-res detect, HD classify
Similarity Metric	Cosine distance
GPU Policy	Fixed-shape inference
Crop Budget	Max 2 per frame

14. System Strengths

- Works with extremely small datasets
 - Efficient on embedded hardware
 - Separates localization from recognition
 - Supports real-time UAV inference
 - Easily replaceable components
 - Robust against terrain variability
 - Minimal GPU memory usage
-

15. Known Limitations

- Strong dependence on seed image quality
 - Sensitive to severe lighting shifts
 - YOLO may miss very small distant features
 - ProtoNet accuracy depends on embedding quality
 - Requires careful threshold tuning
-

16. Future Extensions

Possible upgrades:

- temporal tracking
 - SLAM-assisted feature persistence
 - transformer encoders
 - CLIP-style embeddings
 - active viewpoint selection
 - self-supervised terrain pretraining
 - multi-frame confirmation logic
-

17. Final Architecture Summary

The final system uses:

YOLOv8n
+
Depth Filtering
+
ProtoNet Few-Shot Classification
+
TensorRT FP16 Embedded Inference

to create a lightweight autonomous terrain recognition system optimized for:

- low-data learning
- real-time UAV deployment
- embedded GPU constraints
- fully autonomous operation
- ISRO ASCEND mission requirements